

Assignment2

Youssef Amin and Valerie Barker

February 2026

Introduction

In this assignment, we study the implementation and performance of linear Support Vector Machines (SVMs). We first implement a LinearSVC classifier using stochastic gradient descent with hinge based loss and L2 regularization. We then construct a procedure to generate linearly separable datasets of varying sizes and dimensions in order to analyze scalability. Finally, we compare our implementation with the primal and dual formulations provided by sklearn to examine differences in runtime and convergence behavior.

1 Task 1: LinearSVC Implementation

We implemented a linear Support Vector Classifier using stochastic gradient descent (SGD). The classifier optimizes the objective with hinge loss and L2 regularization.

The weight vector is initialized randomly and updated sample by sample for each epoch. If a sample satisfies the margin condition $y_i(w^T x_i + b) \geq 1$, only the regularization term is applied. Otherwise, the weights and bias are updated using the hinge loss gradient scaled by the regularization parameter C .

The implementation also tracks classification errors, margin violations, and the total loss at each epoch to analyze convergence behavior.

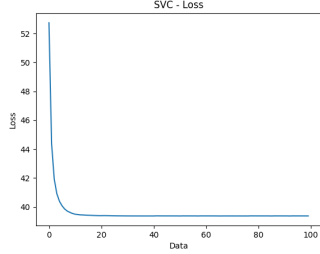


Figure 1: *
Loss Convergence

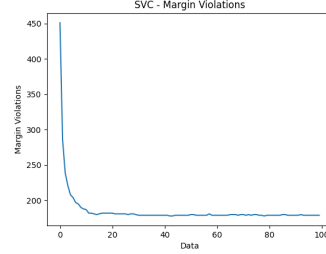


Figure 2: *
Margin Violations



Figure 3: *
Class Errors

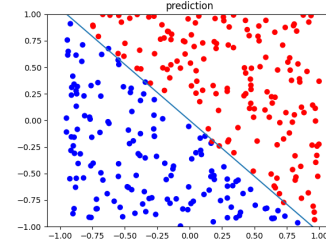


Figure 4: *
Decision Boundary

Figure 5: Training behavior and learned decision boundary of the implemented LinearSVC.

The loss decreases rapidly during the initial epochs and stabilizes as the model approaches convergence. Similarly, the number of margin violations decreases significantly. The final decision boundary closely matches the true separating plane, demonstrating that the SGD based optimization successfully learned a linear separator.

2 Task 2: Linearly Separable Data Generator

We generate linearly separable data by first creating a random vector a that defines a separating hyperplane $a^T x = 0$. We then sample n points uniformly from the range $[-u, u]^d$. Each point is labeled according to the sign of $a^T x$: points on one side of the hyperplane are labeled -1 , and points on the other side are labeled 1 .

The dataset is split into 70% training data and 30% testing data. Because the labels are assigned using the separating hyperplane, the data is linearly separable by construction.

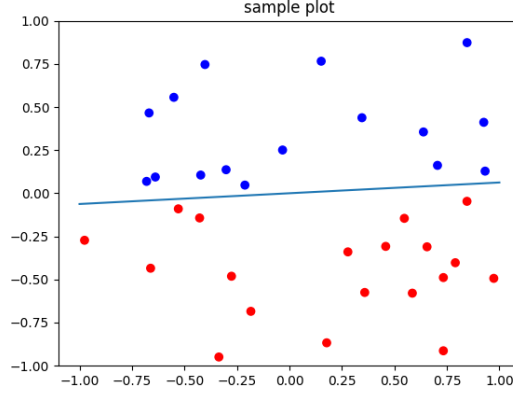


Figure 6: Example dataset generated with $d = 2$ and $n = 100$.

3 Task 3: Scalability Analysis

We evaluated the scalability of our SGD based LinearSVC by training it on nine synthetic linearly separable datasets with dimensions $d \in \{10, 50, 100\}$ and sample sizes $n \in \{500, 5000, 50000\}$. For all runs, we used $u = 100$, learning rate $\eta = 10^{-6}$, 50 epochs, and $C = 200$. For each dataset, we recorded the training runtime and the final loss value.

3.1 Time Cost and Final Loss

d	n	runtime (s)	final loss
10	500	0.19	7.04
10	5000	1.59	5.47
10	50000	14.42	10.20
50	500	0.16	1.08
50	5000	1.57	35.04
50	50000	14.11	28.19
100	500	0.21	2.02
100	5000	1.41	54.34
100	50000	15.08	98.17

Table 1: Training runtime and final loss of LinearSVC across dataset scales.

The runtime increases significantly as the number of samples n grows, while changes in dimension d have a smaller impact. This is expected since the SGD implementation performs updates for each training sample during every epoch, making the algorithm scale approximately linearly with respect to n .

The final loss generally increases for larger datasets and higher dimensions. Because the number of epochs is fixed at 50, larger datasets require more passes to fully minimize the objective. As a result, the model does not fully converge for the largest configurations.

3.2 Loss Convergence

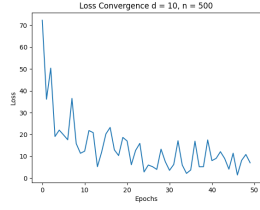


Figure 7: *
 $d = 10, n = 500$

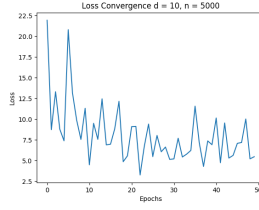


Figure 8: *
 $d = 10, n = 5000$

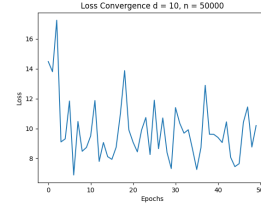


Figure 9: *
 $d = 10, n = 50000$

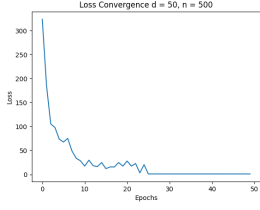


Figure 10: *
 $d = 50, n = 500$

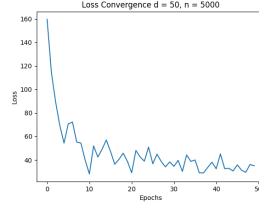


Figure 11: *
 $d = 50, n = 5000$

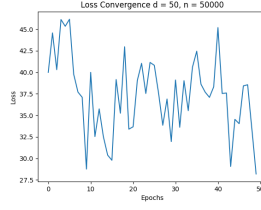


Figure 12: *
 $d = 50, n = 50000$

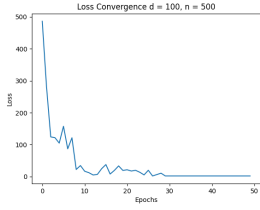


Figure 13: *
 $d = 100, n = 500$

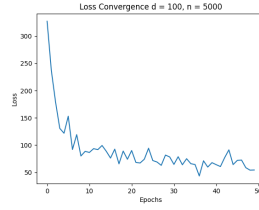


Figure 14: *
 $d = 100, n = 5000$

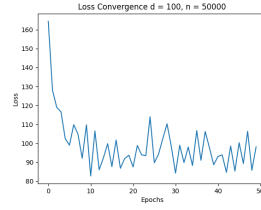


Figure 15: *
 $d = 100, n = 50000$

Figure 16: Loss convergence for all dataset configurations.

Across all configurations, the loss converges quickly during the first few epochs and then levels off. When the number of samples increases, convergence becomes slower because we keep the number of training epochs fixed at 50. Larger

datasets require more passes to fully minimize loss. We also observe fluctuations in the curves, which are expected when using stochastic gradient descent.

4 Task 4: Primal vs Dual Comparison

4.1 Training Time Comparison

In our experiments, the primal formulation was implemented using `dual=False` with `loss="squared_hinge"`, since sklearn does not allow `dual=False` with the standard hinge loss. The dual formulation was implemented using `dual=True` with `loss="hinge"`. Therefore, the comparison reflects both differences in optimization formulation and the associated loss functions supported by sklearn.

d	n	Primal Time (s)
10	500	0.0043
10	5000	0.0040
10	50000	0.0453
50	500	0.0060
50	5000	0.0761
50	50000	0.2933
100	500	0.0020
100	5000	0.1203
100	50000	0.2964

Table 2: Training time of sklearn LinearSVC using primal formulation.

d	n	Dual Time (s)
10	500	0.0019
10	5000	0.0078
10	50000	0.0455
50	500	0.0080
50	5000	0.0201
50	50000	0.3341
100	500	0.0100
100	5000	0.0413
100	50000	0.5478

Table 3: Training time of sklearn LinearSVC using dual formulation.

For small datasets ($n = 500$), both primal and dual formulations train very quickly, with minimal differences in runtime. As the number of samples increases, the dual formulation tends to become slower, especially for higher dimensions. In particular, when n is large relative to d , the primal formulation generally performs better. This aligns with theoretical guidance that the primal

problem is often more efficient when the number of samples is much larger than the number of features.

4.2 Loss Convergence Comparison

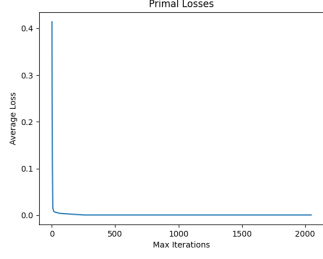


Figure 17: *
Primal ($d = 10, n = 500$)

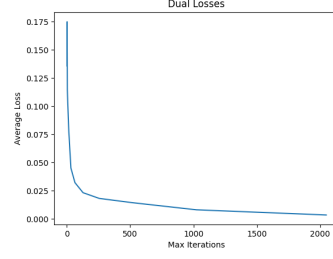


Figure 18: *
Dual ($d = 10, n = 500$)

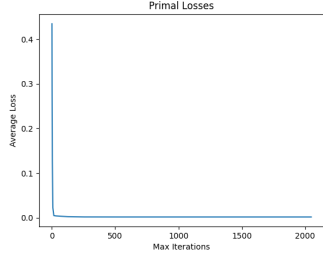


Figure 19: *
Primal ($d = 50, n = 5000$)

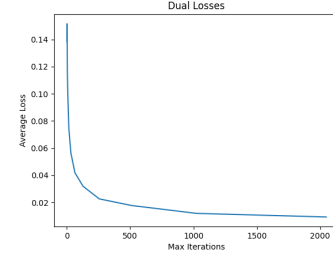


Figure 20: *
Dual ($d = 50, n = 5000$)

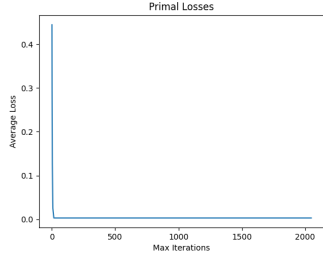


Figure 21: *
Primal ($d = 100, n = 50000$)

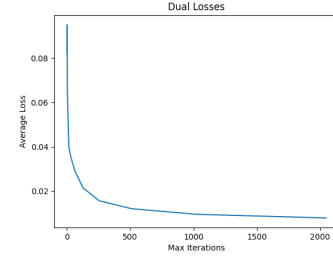


Figure 22: *
Dual ($d = 100, n = 50000$)

Figure 23: Comparison of primal and dual loss convergence for representative dataset scales.

Both formulations converge smoothly as the maximum number of iterations increases. The primal formulation typically decreases more sharply in early iterations, while the dual formulation shows a more gradual reduction. For larger datasets, convergence remains stable but may require more iterations to fully minimize the objective.

The remaining convergence plots for all dataset combinations are available in the project directory under the `images/` folder.

5 Conclusion

Our experiments showed that the runtime of the SGD implementation grows primarily with the number of samples, which is expected given that updates are performed per training example. Convergence behavior also depended on dataset size, as larger datasets required more iterations to fully minimize the objective when the number of epochs was fixed.

When comparing the primal and dual formulations in `sklearn`, we observed that the primal formulation generally performed better when the number of samples was significantly larger than the number of features. Both formulations converged reliably, but their runtime and convergence characteristics differed slightly due to differences in optimization strategy and loss formulation. Overall, the results aligned well with theoretical expectations and provided practical insight into how linear SVMs behave under different scaling regimes.

Project Files

- `LinearSVC.py` - Implementation of the linear SVM using stochastic gradient descent.
- `make_classification.py` - Generates linearly separable synthetic datasets with configurable dimension and size.
- `scalability.py` - Runs scalability experiments and records runtime and loss across dataset scales.
- `compare_sklearn.py` - Compares primal and dual formulations of `sklearn`'s `LinearSVC`.
- `test_LinearSVC.py` - Demonstrates and visualizes the behavior of the implemented classifier.
- `MarginBand.py` - Utility for visualizing the decision boundary and margin band in 2D.
- `test_make_classification.py` - Simple visualization script for generated 2D datasets.